

Optimización y Consultas Federadas con Trino

Integrantes del grupo

- Adrián Blasco Lozano
 - Enric Gil Baquero
 - Hugo Romero Casado
 - Antonio Sala Llaudis
-

1. Arquitectura del entorno

Para esta práctica se desplegó un clúster distribuido de Trino compuesto por:

- 1 Coordinator
- 2 Workers
- PostgreSQL como fuente relacional
- Hive + Parquet como Data Lake analítico

El objetivo fue demostrar cómo Trino permite ejecutar consultas SQL federadas sobre múltiples fuentes heterogéneas sin mover físicamente los datos.

Componentes principales

Coordinator

Se encarga de:

- recibir las consultas SQL,
- generar el plan de ejecución,
- dividir el trabajo en fragments y splits,
- distribuir las tareas entre los workers.

Workers

Procesan las tareas en paralelo para mejorar el rendimiento de las consultas distribuidas.

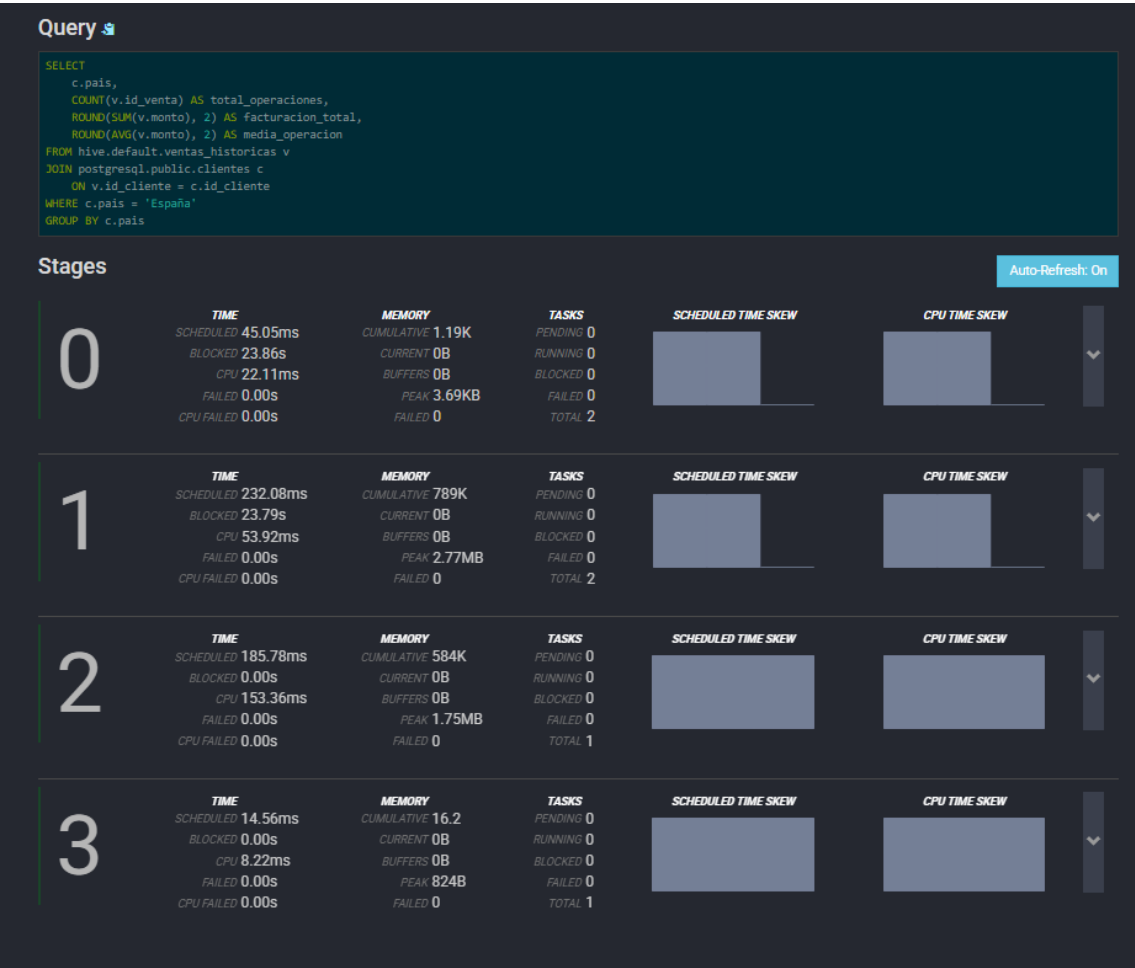
PostgreSQL

Se utiliza como fuente de datos transaccional.

Hive + Parquet

Se utiliza como Data Lake analítico para almacenar grandes volúmenes de información en formato columnar Parquet.

Arquitectura y cluster distribuido



2. Catálogos y conectores configurados

Se configuraron los siguientes catálogos en Trino:

Catálogo	Tipo	Función
postgresql	Relacional	Datos transaccionales
hive	Data Lake	Archivos Parquet
system	Interno	Monitorización

Verificación de catálogos

```
trino> SHOW CATALOGS;
Catalog
-----
hive
postgresql
system
(3 rows)

Query 20260512_152246_00000_h8x2v, FINISHED, 2 nodes
Splits: 36 total, 36 done (100.00%)
1.15 [0 rows, 0B] [0 rows/s, 0B/s]
```

Verificación PostgreSQL

```
trino> SELECT * FROM postgresql.public.clientes;
id_cliente | nombre | pais | score_credito
-----+-----+-----+-----
1 | Empresa de Prueba 1 | España | 700
2 | Empresa de Prueba 2 | México | 650
(2 rows)

Query 20260512_152314_00003_h8x2v, FINISHED, 1 node
Splits: 1 total, 1 done (100.00%)
0.33 [2 rows, 0B] [6 rows/s, 0B/s]
```

Verificación Hive / Parquet

```
trino> SELECT * FROM hive.default.ventas_historicas LIMIT 10;
id_venta | id_cliente | monto | fecha
-----+-----+-----+-----
1 | 628 | 754.65 | 2025-01-01 00:00:00.000
2 | 703 | 955.06 | 2025-01-01 00:01:00.000
3 | 574 | 875.61 | 2025-01-01 00:02:00.000
8 | 92 | 649.3 | 2025-01-01 00:07:00.000
9 | 982 | 161.54 | 2025-01-01 00:08:00.000
10 | 526 | 916.52 | 2025-01-01 00:09:00.000
4 | 768 | 11.5 | 2025-01-01 00:03:00.000
5 | 483 | 226.51 | 2025-01-01 00:04:00.000
6 | 519 | 215.66 | 2025-01-01 00:05:00.000
7 | 155 | 418.96 | 2025-01-01 00:06:00.000
(10 rows)

Query 20260512_152408_00006_h8x2v, FINISHED, 1 node
Splits: 18 total, 18 done (100.00%)
2.18 [15 rows, 2.06MB] [6 rows/s, 969KB/s]
```

3. Consulta federada y análisis distribuido

La consulta federada combina información almacenada en PostgreSQL con datos almacenados en archivos Parquet utilizando Trino como motor unificado.

Query SQL utilizada

```
SELECT
  c.pais,
  COUNT(v.id_venta) AS total_operaciones,
  ROUND(SUM(v.monto), 2) AS facturacion_total,
  ROUND(AVG(v.monto), 2) AS media_operacion
FROM hive.default.ventas_historicas v
JOIN postgresql.public.clientes c
  ON v.id_cliente = c.id_cliente
GROUP BY c.pais
ORDER BY facturacion_total DESC;
```

La consulta realiza:

- JOIN federado,
- COUNT(),
- SUM(),
- AVG(),
- GROUP BY,
- ORDER BY.

Resultado consulta federada

```
trino> SELECT
->   c.pais,
->   COUNT(v.id_venta) AS total_operaciones,
->   ROUND(SUM(v.monto), 2) AS facturacion_total,
->   ROUND(AVG(v.monto), 2) AS media_operacion
-> FROM hive.default.ventas_historicas v
-> JOIN postgresql.public.clientes c
->   ON v.id_cliente = c.id_cliente
-> GROUP BY c.pais
-> ORDER BY facturacion_total DESC;
 pais | total_operaciones | facturacion_total | media_operacion
-----+-----+-----+-----
Espa<C3><B1>a |          96 |      49139.61 |      511.87
M<C3><A9>xico |          88 |      48824.76 |      554.83
(2 rows)
```

Query 20260512_152438_00007_h8x2v, FINISHED, 2 nodes
Splits: 197 total, 197 done (100.00%)
2.16 [100K rows, 2.06MB] [46.4K rows/s, 980KB/s]

4. Optimización, Pushdown y EXPLAIN ANALYZE

Se utilizaron las herramientas EXPLAIN y EXPLAIN ANALYZE para analizar cómo Trino distribuye la ejecución de la consulta.

El motor divide automáticamente la query en:

- fragments,
- stages,
- splits.

Estas tareas son distribuidas entre los workers del clúster para ejecutar el procesamiento en paralelo.

EXPLAIN ANALYZE

```
trino> EXPLAIN ANALYZE
-> SELECT
->   c.pais,
->   COUNT(v.id_venta) AS total_operaciones,
->   ROUND(SUM(v.monto), 2) AS facturacion_total,
->   ROUND(AVG(v.monto), 2) AS media_operacion
-> FROM hive.default.ventas_historicas v
-> JOIN postgresql.public.clientes c
->   ON v.id_cliente = c.id_cliente
-> GROUP BY c.pais
-> ORDER BY facturacion_total DESC;

Query Plan

Trino version: 451
Queued: 261.36us, Analysis: 304.37ms, Planning: 50.26ms, Execution: 755.39ms
Fragment 1 [ROUND_ROBIN]
CPU: 18.66ms, Scheduled: 27.45ms, Blocked 16.37s (Input: 15.43s, Output: 0.00ns), Input: 2 rows (788); per task: avg.: 1.00 std.dev.: 0.00, Output: 2
Output layout: [pais, count, expr, expr_1]
Output partitioning: SINGLE []
LocalMerge[orderBy = [expr DESC NULLS LAST]]
<E2><94><82> Layout: [pais:varchar(50), count:bigint, expr:double, expr_1:double]
<E2><94><82> Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
<E2><94><82> CPU: 2.00ms (0.87%), Scheduled: 5.00ms (1.10%), Blocked: 951.00ms (1.42%), Output: 2 rows (788)
<E2><94><82> Input avg.: 0.06 rows, Input std.dev.: 387.30%
<E2><94><94><E2><94><80> PartialSort[orderBy = [expr DESC NULLS LAST]]
<E2><94><82> Layout: [pais:varchar(50), count:bigint, expr:double, expr_1:double]
<E2><94><82> Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
<E2><94><82> CPU: 2.00ms (0.87%), Scheduled: 5.00ms (1.10%), Blocked: 951.00ms (1.42%), Output: 2 rows (788)
<E2><94><82> Input avg.: 0.06 rows, Input std.dev.: 387.30%
<E2><94><94><E2><94><80> RemoteSource[sourceFragmentIds = [2]]
Layout: [pais:varchar(50), count:bigint, expr:double, expr_1:double]
CPU: 0.00ms (0.00%), Scheduled: 0.00ms (0.00%), Blocked: 15.43s (23.08%), Output: 2 rows (788)
Input avg.: 0.06 rows, Input std.dev.: 387.30%

Fragment 2 [HASH]
CPU: 40.77ms, Scheduled: 112.76ms, Blocked 25.83s (Input: 12.28s, Output: 0.00ns), Input: 4 rows (1968); per task: avg.: 2.00 std.dev.: 2.00, Output:
Amount of input data processed by the workers for this stage might be skewed
Output layout: [pais, count, expr, expr_1]
Output partitioning: ROUND_ROBIN []
Project[]
<E2><94><82> Layout: [pais:varchar(50), count:bigint, expr:double, expr_1:double]
<E2><94><82> Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
<E2><94><82> CPU: 15.00ms (6.55%), Scheduled: 28.00ms (6.17%), Blocked: 0.00ms (0.00%), Output: 2 rows (788)
<E2><94><82> Input avg.: 0.06 rows, Input std.dev.: 387.30%
<E2><94><82> expr := round(sum, integer '2')
<E2><94><82> expr_1 := round(avg, integer '2')
<E2><94><94><E2><94><80> Aggregate[type = FINAL, keys = [pais]]
Layout: [pais:varchar(50), count:bigint, sum:double, avg:double]
Estimates: {rows: ? (?), cpu: 0, memory: 0B, network: 0B}
CPU: 9.00ms (3.93%), Scheduled: 39.00ms (8.59%), Blocked: 0.00ms (0.00%), Output: 2 rows (788)
Input avg.: 0.13 rows, Input std.dev.: 387.30%
count := count(count_2)

Query 20260512_152619_00009_h8x2v, FINISHED, 3 nodes
Splits: 197 total, 197 done (100.00%)
1.20 [100K rows, 2.06MB] [83.7K rows/s, 1.73MB/s]
```

5. Benchmark y Predicate Pushdown

Se realizaron pruebas comparando la ejecución:

- sin filtros,
- con filtros WHERE.

Consulta sin filtro

La consulta completa procesó:

- 197 splits,
- aproximadamente 1.20 segundos,
- más volumen de datos.

Consulta optimizada con filtro

```
SELECT
  c.pais,
  COUNT(v.id_venta) AS total_operaciones,
  ROUND(SUM(v.monto), 2) AS facturacion_total,
  ROUND(AVG(v.monto), 2) AS media_operacion
FROM hive.default.ventas_historicas v
JOIN postgresql.public.clientes c
  ON v.id_cliente = c.id_cliente
WHERE c.pais = 'España'
GROUP BY c.pais;
```

Resultado:

- menor número de splits,
- menor tiempo de ejecución,
- menor volumen de procesamiento,
- menor tráfico interno entre workers.

Tiempo aproximado:

- ~0.53 segundos.

Esto demuestra el uso de optimización mediante predicate pushdown.

El filtro WHERE permitió reducir la cantidad de datos procesados antes de realizar el JOIN distribuido, mejorando significativamente el rendimiento de la consulta.

6. Monitorización y ejecución distribuida

La interfaz web de Trino permitió analizar:

- consumo de CPU,
- memoria,
- throughput,

- stages,
- workers activos,
- paralelismo,
- timelines,
- splits distribuidos.

Historial de consultas

QUERY DETAILS						
User, source, query ID, query state, resource group, error name, or query text		State:	Running Queued Finished Failed	Sort	Reorder Interval	Show
20260512_152826_00011_h8x2v		5:28pm	FINISHED			
trino trino-cli global ✓ 162 ▶ 0 ▮ 0 ⌚ 494.41ms ⌚ 531.93ms ⌚ 237.00ms 🗄 0B 📡 4.52MB 📊 1.34M		SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion FROM hive.default.ventas_historicas v ...				
20260512_152715_00010_h8x2v		5:27pm	FINISHED			
trino trino-cli global ✓ 179 ▶ 0 ▮ 0 ⌚ 716.38ms ⌚ 785.77ms ⌚ 219.00ms 🗄 0B 📡 5.40MB 📊 784K		EXPLAIN ANALYZE SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion ...				
20260512_152619_00009_h8x2v		5:26pm	FINISHED			
trino trino-cli global ✓ 197 ▶ 0 ▮ 0 ⌚ 890.62ms ⌚ 1.20s ⌚ 299.00ms 🗄 0B 📡 5.40MB 📊 1.17M		EXPLAIN ANALYZE SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion ...				
20260512_152539_00008_h8x2v		5:25pm	FINISHED			
trino trino-cli global ✓ 1 ▶ 0 ▮ 0 ⌚ 219.19ms ⌚ 581.26ms ⌚ 0.00ms 🗄 0B 📡 5.17KB 📊 0		EXPLAIN SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion ...				
20260512_152438_00007_h8x2v		5:24pm	FINISHED			
trino trino-cli global ✓ 197 ▶ 0 ▮ 0 ⌚ 2.10s ⌚ 2.16s ⌚ 1.31s 🗄 0B 📡 5.40MB 📊 3.36M		SELECT c.pais, COUNT(v.id_venta) AS total_operaciones, ROUND(SUM(v.monto), 2) AS facturacion_total, ROUND(AVG(v.monto), 2) AS media_operacion FROM hive.default.ventas_historicas v ...				
20260512_152408_00006_h8x2v		5:24pm	FINISHED			
trino trino-cli global ✓ 18 ▶ 0 ▮ 0 ⌚ 1.89s ⌚ 2.18s ⌚ 478.00ms 🗄 0B 📡 2.11KB 📊 1.51K		SELECT * FROM hive.default.ventas_historicas LIMIT 10				
20260512_152405_00005_h8x2v		5:24pm	FINISHED			

Query Details

Query

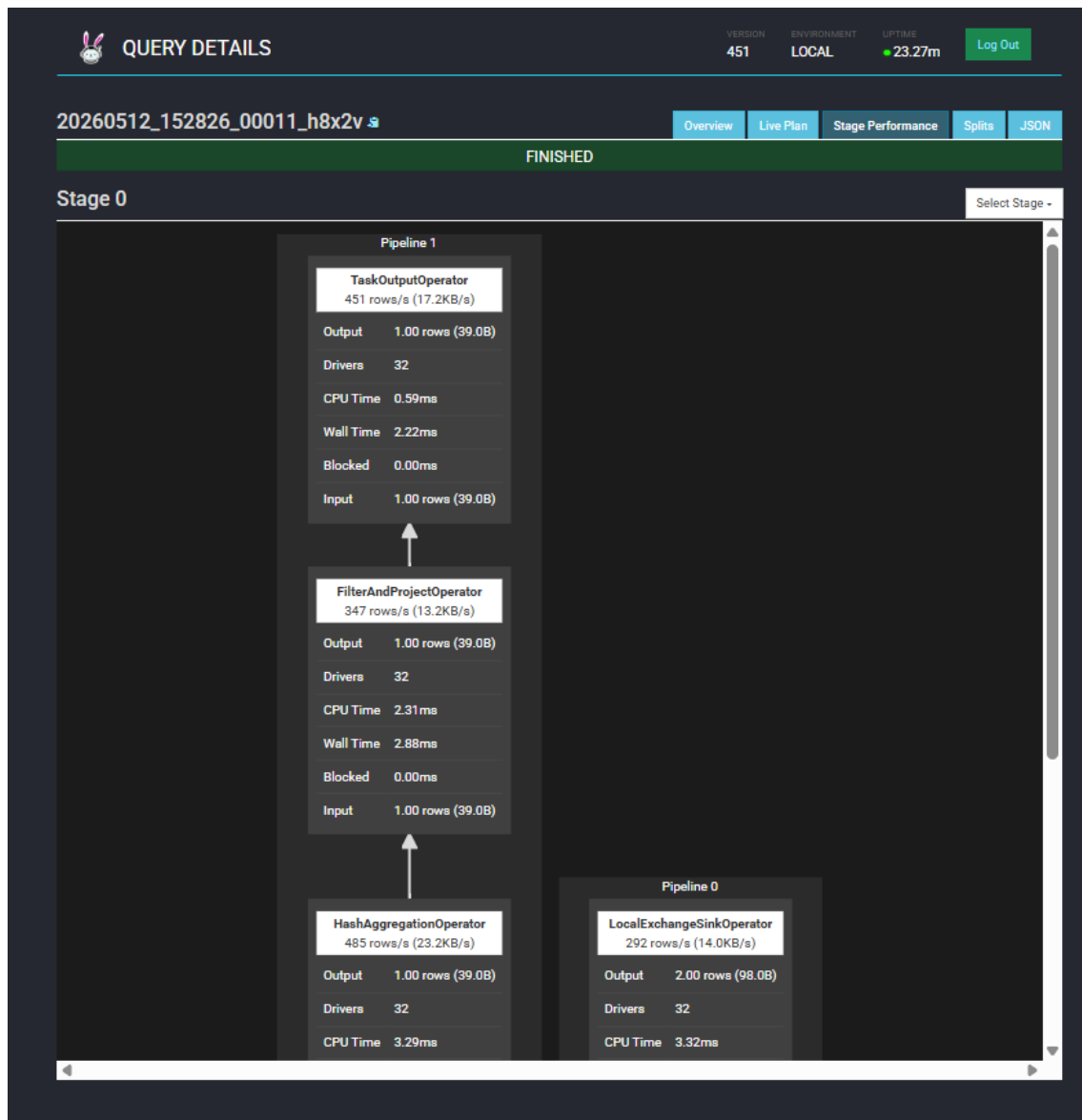
```
SELECT
  c.pais,
  COUNT(v.id_venta) AS total_operaciones,
  ROUND(SUM(v.monto), 2) AS facturacion_total,
  ROUND(AVG(v.monto), 2) AS media_operacion
FROM hive.default.ventas_historicas v
JOIN postgresql.public.clientes c
  ON v.id_cliente = c.id_cliente
WHERE c.pais = 'España'
GROUP BY c.pais
```

Stages

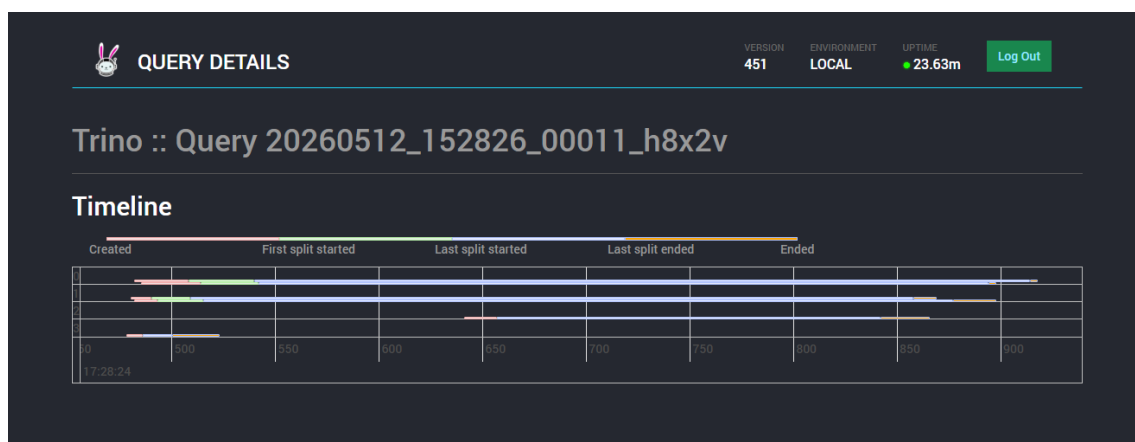
Auto-Refresh: On

	TIME	MEMORY	TASKS	SCHEDULED TIME SKEW	CPU TIME SKEW	
0	SCHEDULED 45.05ms BLOCKED 23.86s CPU 22.11ms FAILED 0.00s CPU FAILED 0.00s	CUMULATIVE 1.19K CURRENT 0B BUFFERS 0B PEAK 3.69KB FAILED 0	PENDING 0 RUNNING 0 BLOCKED 0 FAILED 0 TOTAL 2			▼
1	SCHEDULED 232.08ms BLOCKED 23.79s CPU 53.92ms FAILED 0.00s CPU FAILED 0.00s	CUMULATIVE 789K CURRENT 0B BUFFERS 0B PEAK 2.77MB FAILED 0	PENDING 0 RUNNING 0 BLOCKED 0 FAILED 0 TOTAL 2			▼
2	SCHEDULED 185.78ms BLOCKED 0.00s CPU 153.36ms FAILED 0.00s CPU FAILED 0.00s	CUMULATIVE 584K CURRENT 0B BUFFERS 0B PEAK 1.75MB FAILED 0	PENDING 0 RUNNING 0 BLOCKED 0 FAILED 0 TOTAL 1			▼
3	SCHEDULED 14.56ms BLOCKED 0.00s CPU 8.22ms FAILED 0.00s CPU FAILED 0.00s	CUMULATIVE 16.2 CURRENT 0B BUFFERS 0B PEAK 824B FAILED 0	PENDING 0 RUNNING 0 BLOCKED 0 FAILED 0 TOTAL 1			▼

Stage Performance



Splits y paralelismo



7. Pensamiento crítico

Si una de las fuentes de datos estuviera ubicada en otra región geográfica, la latencia de red aumentaría considerablemente.

Consecuencias:

- mayor tiempo de respuesta,
- incremento del tráfico de red,
- peor rendimiento en joins distribuidos,
- posibles cuellos de botella,
- mayor consumo de recursos.

Por este motivo, en entornos Big Data reales es importante optimizar:

- la ubicación de los datos,
- el uso de cachés,
- el pushdown,
- la distribución de cargas.

8. Conclusión

La práctica permitió demostrar cómo Trino puede actuar como un motor SQL federado capaz de consultar múltiples fuentes heterogéneas en tiempo real sin necesidad de mover físicamente los datos.

Además, se comprobó:

- la ejecución distribuida,
- el paralelismo mediante splits,
- la optimización mediante predicate pushdown,
- y la monitorización del rendimiento desde la interfaz web de Trino.